LAB 4: Ingress Controller

Getting Started with Ingress Controller

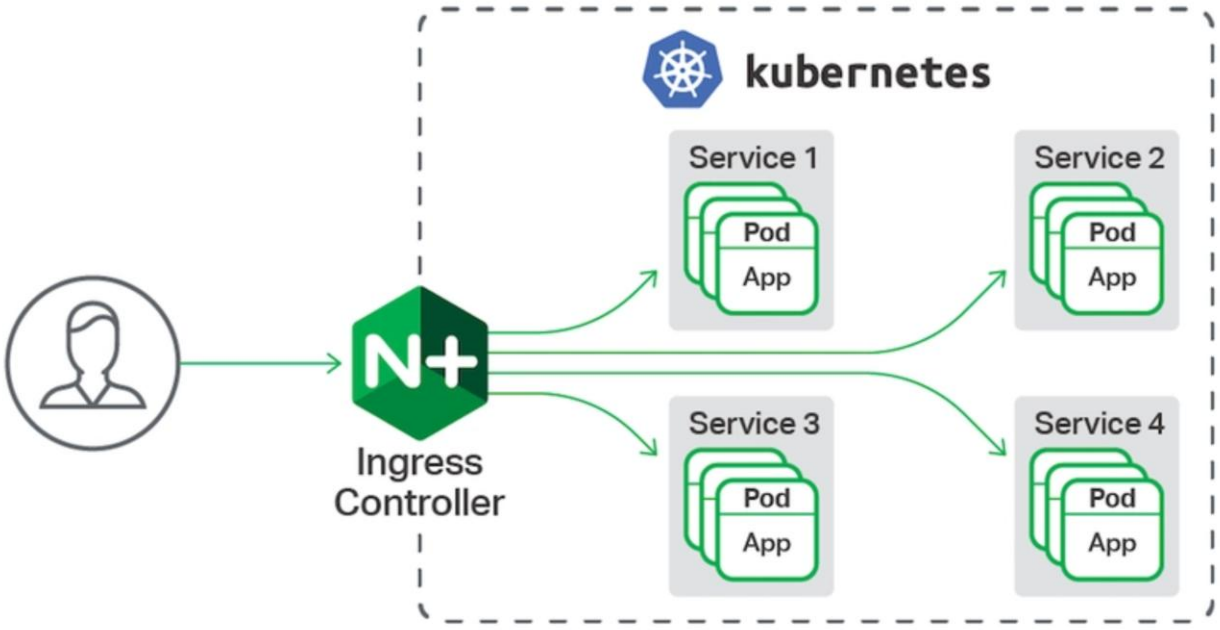
In previous labs, we containerized our application using Docker and used NGINX to manage load balancing and route requests between containers. Now that our application is deployed on Kubernetes, we need a more integrated way to manage external traffic coming into the cluster — this is where Ingress comes in.

In simple terms, "ingress" just means "entry." In the Kubernetes world:

- *Ingress* is a special Kubernetes object (like Deployments or Services) that defines routing rules for HTTP/HTTPS traffic coming from outside the cluster.
- An *Ingress Controller* (like NGINX) is the actual tool running inside the cluster that reads those rules and makes them work — it directs external traffic to the right services based on path or domain name.

In this lab, we'll use Ingress and an Ingress Controller to expose our application to the outside world through a clean and centralized entry point.

Ingress Controller in Kubernetes:



Let's get started!

First, delete the Nginx service folder and the docker-compose.yaml file as we no longer need them.

Kubernetes natively supports Ingress resources, allowing you to define routing and load-balancing rules using a simple YAML file. These rules tell Kubernetes how to direct external traffic to the appropriate services inside your cluster.

LAB 4: Ingress Controller

To use NGINX as the Ingress Controller, you can enable it with the following command:

minikube addons enable ingress

```
C:\Windows\System32>minikube addons enable ingress
* ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
* After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
  - Using image registry.k8s.io/ingress-nginx/controller:v1.11.3
  - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.4.4
  - Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.4.4
* Verifying ingress addon...
* The 'ingress' addon is enabled

C:\Windows\System32>
```

Check if the ingress controller is running:

kubectl get pods -n ingress-nginx

```
(venv) C:\Users\joudy\Desktop\TaskApplication_JoudySabbagh>kubectl get pods -n ingress-nginx
NAME                                        READY   STATUS      RESTARTS      AGE
ingress-nginx-admission-create-2hdn8        0/1     Completed   0             15h
ingress-nginx-admission-patch-g7slw         0/1     Completed   0             15h
ingress-nginx-controller-56d7c84fd4-mjt4r   1/1     Running     1 (27m ago)   15h
```

Screenshot the output for submission

Now that the Ingress controller is up and running, let's define the routing rules it should follow. Create a file named *ingress.yaml* and paste the following configuration into it:

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: taskapp-ingress
spec:
  rules:
  - host: taskapp.local
    http:
      paths:
      - path: /api/
        pathType: Prefix
        backend:
          service:
            name: backend-service
            port:
              number: 5000
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
```

The Ingress is configured to route /api/ requests to the backend service, and all other requests (/) to the frontend service. Additionally, we've named the Ingress resource taskapp-ingress and Ingress host taskapp.local, which we'll use to access the application in the browser.

LAB 4: Ingress Controller

Run the following command to apply the file:

kubectl apply -f ingress.yaml

```
(venv) C:\Users\joudy\Desktop\TaskApplication_JoudySabbagh>kubectl apply -f ingress.yaml
ingress.networking.k8s.io/taskapp-ingress created
```

Let's verify that our Ingress is properly configured and actively routing requests. Run the following:

kubectl describe ingress taskapp-ingress

```
(venv) C:\Users\joudy\Desktop\TaskApplication_JoudySabbagh>kubectl describe ingress taskapp-ingress
Name:             taskapp-ingress
Labels:           <none>
Namespace:        default
Address:          192.168.49.2
Ingress Class:    nginx
Default backend:  <default>
Rules:
  Host           Path  Backends
  ----           ----  --------
  taskapp.local
                 /api/   backend-service:5000 (10.244.0.4:5000,10.244.0.5:5000)
                 /       frontend-service:80 (10.244.0.6:3000,10.244.0.7:3000)
Annotations:     <none>
Events:
  Type    Reason  Age                From                      Message
  ----    ------  ----               ----                      -------
  Normal  Sync    38m (x3 over 48m)  nginx-ingress-controller  Scheduled for sync

(venv) C:\Users\joudy\Desktop\TaskApplication_JoudySabbagh>
```

Screenshot the output for submission

To check how the Ingress controller is exposed, run:

kubectl get svc -n ingress-nginx

```
(venv) C:\Users\joudy\Desktop\TaskApplication_JoudySabbagh>kubectl get svc -n ingress-nginx
NAME                                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)                      AGE
ingress-nginx-controller             NodePort    10.109.3.109   <none>        80:30168/TCP,443:31761/TCP   15h
ingress-nginx-controller-admission   ClusterIP   10.104.23.13   <none>        443/TCP                      15h
```

Screenshot the output for submission

This command shows the type of service used by the Ingress controller, along with its Cluster IP, External IP, and the ports it's using to handle traffic.

LAB 4: Ingress Controller

In the current setup:

- The Ingress controller is of type NodePort.
- It has no External IP assigned.
- It exposes traffic on a high port (30168) that must be manually opened on your computer, which is not ideal.

Note: To use a NodePort service, we would need to manually open a specific high-numbered port on our machine and access the app through that port. This approach isn't ideal for production because:

- It exposes random high ports
- It requires manual port management
- It doesn't support direct access through a domain name or standard ports like 80 and 443

To solve this, we'll switch the service type to LoadBalancer. This allows Kubernetes (with Minikube tunnel) to automatically assign an external IP and expose the app on standard ports like 80 and 443 — making it cleaner, more accessible, and better suited for production-like environments.

Run the following

kubectl patch svc ingress-nginx-controller -n ingress-nginx --type=merge -p "{\"spec\": {\"type\": \"LoadBalancer\"}}"

Then check the Ingress controller type of service.

```
(venv) C:\Users\joudy\Desktop\TaskApplication_JoudySabbagh>kubectl patch svc ingress-nginx-controller -n ingress-nginx --type=merge -p "{\"spec\": {\"type\": \"LoadBalancer\"}}"
service/ingress-nginx-controller patched

(venv) C:\Users\joudy\Desktop\TaskApplication_JoudySabbagh>kubectl get svc -n ingress-nginx
NAME                                 TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)                      AGE
ingress-nginx-controller             LoadBalancer   10.109.3.109    127.0.0.1     80:30168/TCP,443:31761/TCP   15h
ingress-nginx-controller-admission   ClusterIP      10.104.23.13    <none>        443/TCP                      15h
```

Your Ingress controller service is now of type LoadBalancer, and it has an EXTERNAL-IP of 127.0.0.1.

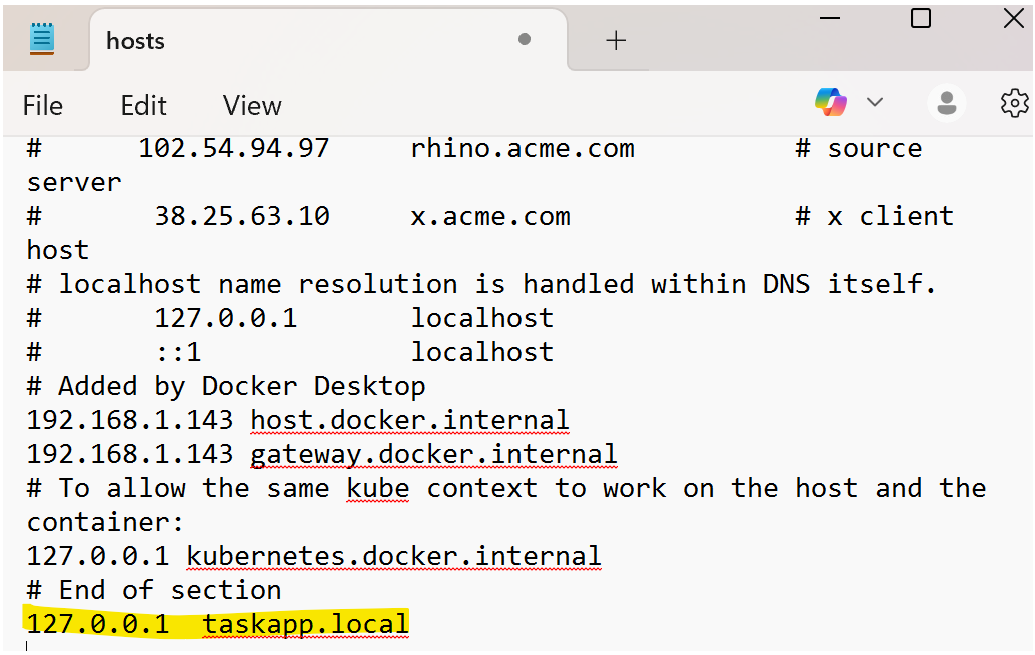
Screenshot the output for submission

To link the external IP to the taskapp.local name, we need to edit the etc\hosts file, which stores the mappings between hostnames and IP addresses.

Go to *C:\Windows\System32\drivers\etc\hosts*, open the file as an **administrator**, and add the following entry:

127.0.0.1 taskapp.local

LAB 4: Ingress Controller

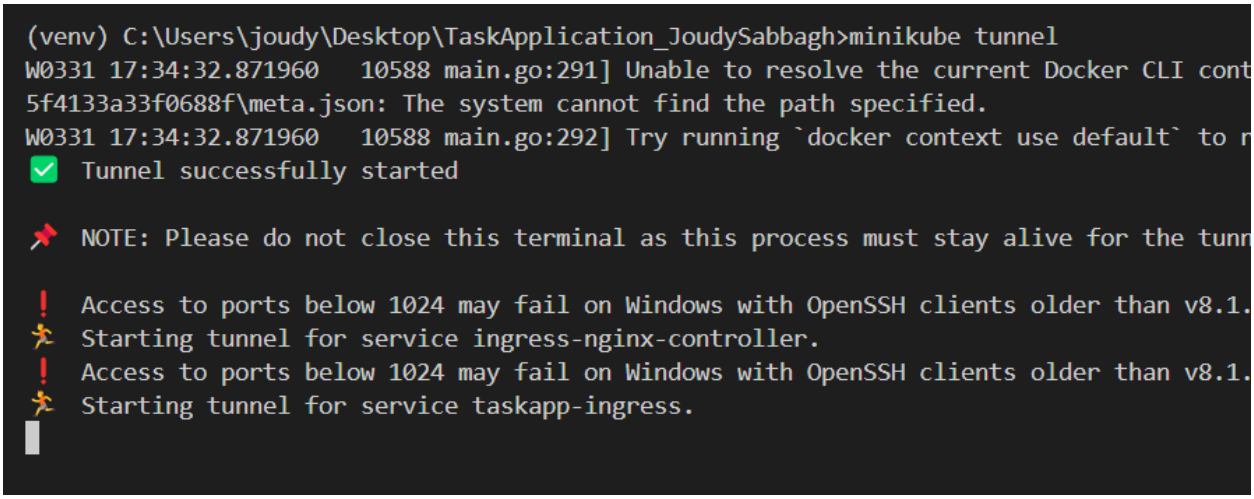


Save and exit the file.

Screenshot the modified file for submission

Now, start the Minikube tunnel to establish a connection between your local machine (host) and the Minikube cluster:

minikube tunnel



Open http://taskapp.local//. You should see your task manager app up and running.

Try adding tasks—you'll notice a problem: the newly added elements don't always appear right away. Sometimes they show up after refreshing and then disappear again.

LAB 4: Ingress Controller

This happens because we're running two backend pods, each with its own separate SQLite database, so the data isn't shared between them.

To fix this issue, we need to switch to a centralized database like PostgreSQL, which can be accessed by both backend pods, ensuring consistent and shared data across the application.

In your backend service, replace the database configuration with the following:

app.config['SQLALCHEMY_DATABASE_URI'] = 'postgresql://postgres:postgres@postgres-service:5432/tasks'

Rebuild and push the updated backend service image on the docker hub.

```
(venv) C:\Users\joudy\Desktop\TaskApplication_JoudySabbagh>docker build -t joudy123/service_backend ./Backend
[+] Building 1.6s (11/11) FINISHED
 => [internal] load build definition from dockerfile
 => => transferring dockerfile: 469B
 => [internal] load metadata for docker.io/library/python:3.9-alpine
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/5] FROM docker.io/library/python:3.9-alpine@sha256:e345f1410de8c8c40a0afac784deabce796a52f26965c41290a710d4fb47fabe
 => [internal] load build context
 => => transferring context: 1.44kB
 => CACHED [2/5] WORKDIR /app
 => CACHED [3/5] COPY requirements.txt /app/requirements.txt
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
 => [5/5] COPY . /app
 => exporting to image
 => => exporting layers
 => => writing image sha256:e9143cdf67c77eda5c9a89db479ffd35130d1126fb1987f4bc254a73054415d0
 => => naming to docker.io/joudy123/service_backend

What's Next?
  View a summary of image vulnerabilities and recommendations → docker scout quickview

(venv) C:\Users\joudy\Desktop\TaskApplication_JoudySabbagh>docker push joudy123/service_backend
Using default tag: latest
The push refers to repository [docker.io/joudy123/service_backend]
2c340785d9fe: Pushed
0e42c8048393: Layer already exists
babb233b4ce3: Layer already exists
3bc3f41489a0: Layer already exists
efd2d69eb095: Layer already exists
4f7cd6c868d5: Layer already exists
```

Update the "backend-deployment.yaml" file to use a PostgreSQL database instead of SQLite:

```
        env:
        - name: FLASK_ENV
          value: "development"
        - name: DATABASE_URL
          value: "postgresql://postgres:postgres@postgres-service:5432/tasks"
```

LAB 4: Ingress Controller

Now we need to create a “persistent-volume.yaml” file because the PostgreSQL database requires **persistent storage**. This ensures that the data is not lost when the pod restarts or gets rescheduled. The Persistent Volume provides a stable storage location outside the pod's lifecycle, allowing PostgreSQL to store and access data reliably.

Create the file and copy the following code:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: taskapp-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  hostPath:
    path: /mnt/data/taskapp

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: taskapp-pvc
spec:
  accessModes:
   - ReadWriteMany
  resources:
    requests:
     storage: 500Mi
```

This file creates a *Persistent Volume* (a piece of storage on the host machine) and a *Persistent Volume Claim* (a request from a pod to use that storage). It allows your app to save data in a shared folder (/mnt/data/taskapp) that won’t be lost if the pod restarts.

We also need to add a configuration file called “postgres-deployment.yaml”, which will define how the PostgreSQL database is deployed. This file tells Kubernetes to run a PostgreSQL container, connect it to the persistent volume for data storage, and make it available as a service for other pods (like the backend) to access.

Create this configuration file and copy the following code:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
    - ReadWriteOnce                         # Only one pod can write to this volume at a time
  resources:
    requests:
      storage: 1Gi

apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
spec:
  selector:
    matchLabels:
      app: postgres
```

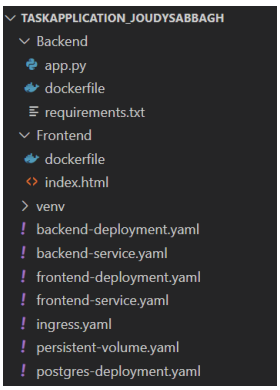
```
  replicas: 1                                   # Only one database instance
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:14
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
              value: tasks
            - name: POSTGRES_USER
              value: postgres
            - name: POSTGRES_PASSWORD
              value: postgres
          volumeMounts:
            - name: postgres-storage
              mountPath: /var/lib/postgresql/data
      volumes:
        - name: postgres-storage
          persistentVolumeClaim:
            claimName: postgres-pvc

apiVersion: v1
kind: Service
metadata:
  name: postgres-service
spec:
  selector:
    app: postgres
  ports:
    - port: 5432
      targetPort: 5432
```

This YAML sets up:

- A PostgreSQL pod with persistent storage.
- A volume that requests 1Gi of space, ensuring data is saved even if the pod restarts (ReadWriteOnce allows one pod to use it at a time).
- A service that exposes PostgreSQL internally on port 5432 so other pods (like the backend) can connect to it.

Now, both backend pods share a single, centralized database.



Note: Be sure to apply all the updated and newly added YAML files.

Your directory should look something like this.

LAB 4: Ingress Controller

Then, restart the Minikube tunnel and access your application.

You should now be able to add and remove items without issues — everything should be working as expected!

Task Manager

Enter task | Add Task

- lab 4 Delete

Screenshot the output for submission

Pretty-print ☐


```
[
  {
    "id": 1,
    "title": "lab 4"
  }
]
```


Screenshot the output for submission

LAB 4: Ingress Controller

Submissions:

- A zip file containing your lab code.
- Screenshots that display the successful execution of the Ingress controller commands throughout this lab.

Note: You should include all screenshots from the lab where you find *"📷 Screenshot the output for submission"*